

SmartDialog Chatbot — Technical Architecture & Functional Workflow

Oct 4, 2026 · @Ketan

Context and constraints

SmartDialog is a multilingual RAG chatbot that answers policy questions from documents and payment questions from live data through MCP tools. It must be extensible to more tools and to an email channel without redesign.

Area	Decision / constraint
Language & framework	Java only (no Python in the office); LangChain4j
LLM	Gemma 4 31B, locally hosted; also the eval judge
Embeddings	bge-m3 (dense, 1024-dim, multilingual and cross-lingual)
Store	PostgreSQL + pgvector (vectors) and Postgres full-text search (lexical)
Corpus	1,000+ docs, EN and FR only; inconsistent structure (some standalone, some depend on surrounding context)
Users	Any language
Tools	RetroFees MCP server: 1 tool today, 6-10 planned, payment data across several tables; more MCP servers later
Eval data	Historical Q&A stored in the DB; a few golden datasets
Future	Client servicing email channel; clarifying questions back to the user

Out of scope for evals: application regression testing. Evals measure question-and-answer quality.

Design principles

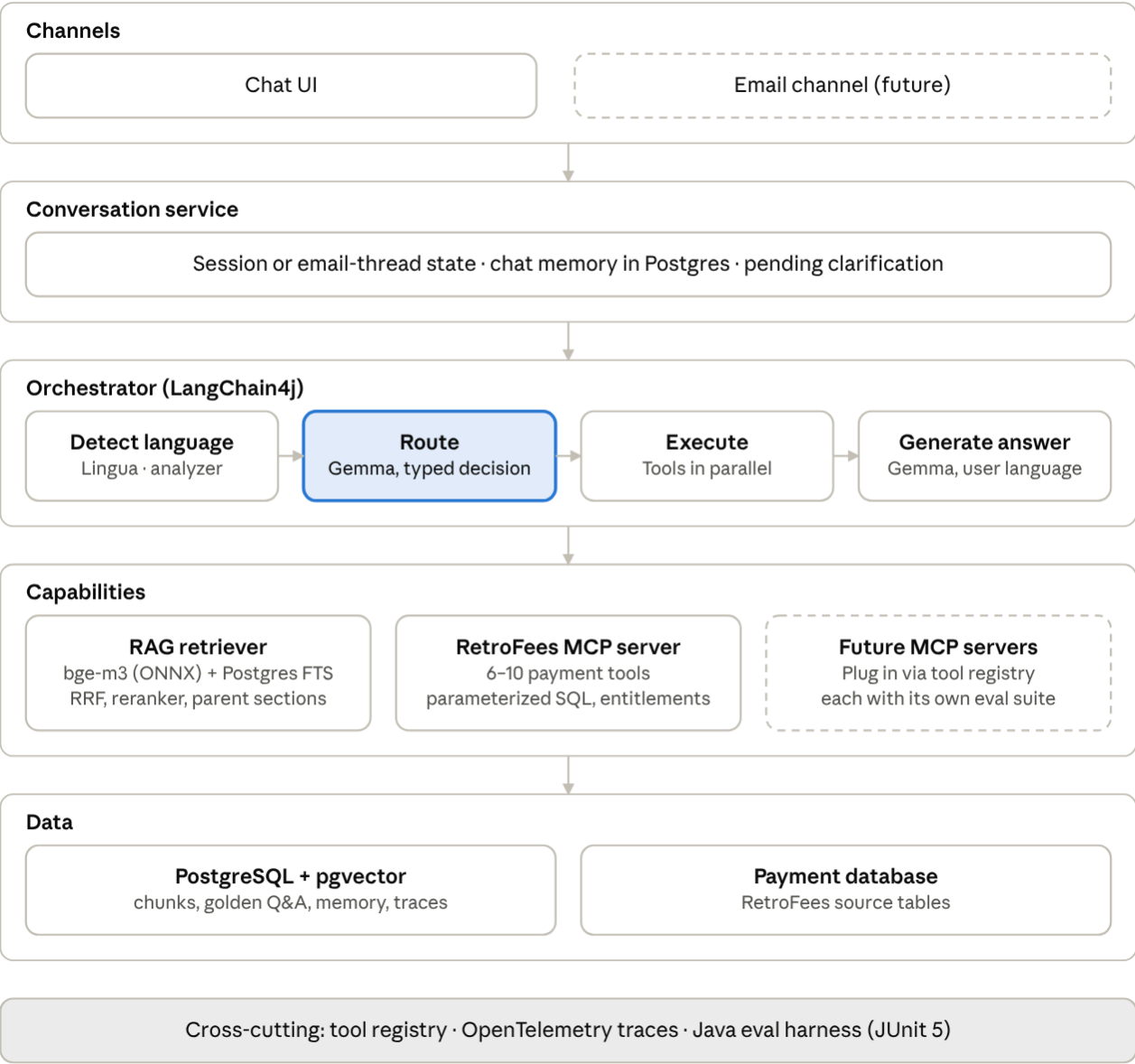
Six rules keep behaviour identical across languages and let new tools plug in without touching the core.

1. **One pipeline for every language.** No per-language branches in retrieval or routing; language only changes the analyzer used and the language of the final answer.
2. **No LLM in the retrieval path unless measured.** Retrieval runs on the user's original query (bge-m3 is cross-lingual), so a bad translation cannot cause a wrong answer.
3. **Channel-agnostic core.** Chat and email are adapters; the orchestrator never knows which channel a question came from.
4. **Structured decisions, not free text.** The router returns a typed `RouteDecision` (route, multi-route, clarify, out of scope) via structured output.
5. **Tools are registered, not hard-coded.** Each tool carries its contract, examples, entitlements and its own eval cases. No eval cases, no release.
6. **Every answer is traceable.** Each turn produces a trace (language, decision, tool calls, chunk IDs, answer) used for debugging, evals and audit.

Technical architecture

Five layers, with the router as the only place that knows which capabilities exist. Channels and capabilities can be added without changing the orchestrator.

One channel-agnostic core serves chat today and email later



technical architecture · 5 layers

Dashed boxes are planned, not built. Every layer writes to the turn trace that the eval harness reads.

Component	Java building blocks
Orchestrator, router, generator	LangChain4j <code> AiServices </code> , structured output, OpenAI-compatible or Ollama client to local Gemma 4 31B
Language detection	Lingua (JVM)
Embeddings and reranking	LangChain4j <code> OnnxEmbeddingModel </code> (bge-m3), <code> OnnxScoringModel </code> (multilingual reranker)

Component	Java building blocks
Retrieval	pgvector HNSW + Postgres FTS, RRF in SQL, wrapped as a custom <code>ContentRetriever</code>
MCP tools	LangChain4j <code>McpToolProvider</code>
Memory	<code>ChatMemoryProvider</code> with a Postgres-backed <code>ChatMemoryStore</code>
Evals	JUnit 5, LangChain4j judges, optional RAGE4j
Tracing	OpenTelemetry Java agent + custom spans

Retrieval design

Drop LLM query translation entirely: bge-m3 is cross-lingual, so the user's original query retrieves EN and FR documents directly. Keep a small lexical leg that needs no translation, and let evals decide whether it earns its place.

Why not vector-only by default. Dense search is weak on exact tokens: fee codes, product codes, client or account IDs, acronyms, regulation numbers. These tokens are identical in EN and FR, so a language-neutral keyword match catches them without any translation.

Pipeline (same for every language)

1. Detect query language with Lingua (JVM library). Used only for the analyzer choice and the answer language, never to translate.
2. Dense leg: embed the original query with bge-m3; pgvector HNSW cosine search, top 40.
3. Lexical leg, two signals in one SQL query:
 - `tsv_simple` (Postgres `simple` config, no stemming) over all chunks: exact codes, IDs, names.
 - `tsv_lang` (`english` or `french` config per chunk): stemmed match, applied only to chunks in the query's language.
4. Fuse with Reciprocal Rank Fusion in SQL ($k = 60$). Rank-based, so Postgres `ts_rank` not being true BM25 does not matter.
5. Rerank the top 30 with a local multilingual cross-encoder (bge-reranker-v2-m3 via LangChain4j `OnnxScoringModel`), keep top 6–8. If no reranker can be hosted, skip this step rather than use Gemma as a reranker (slow, adds variance).
6. Expand to parent sections (see Ingestion) and pass to generation with chunk IDs for citations.

All numbers above are starting values to tune with evals.

Chunk table (one table, one index version column)

Column	Purpose
<code>chunk_id</code> , <code>doc_id</code> , <code>parent_id</code>	Identity and parent-child expansion
<code>lang</code>	<code>en</code> / <code>fr</code>
<code>doc_type</code>	standalone / part-of-set
<code>title</code> , <code>section_path</code>	Context header and citations
<code>content</code>	Chunk text with context header prepended
<code>embedding_vector(1024)</code>	bge-m3 dense; HNSW index, <code>vector_cosine_ops</code>
<code>tsv_simple</code> , <code>tsv_lang</code>	GIN indexes for the lexical leg
<code>metadata jsonb</code>	Product, date, entitlement tags
<code>index_version</code>	Ties every eval run to an index build

Decide with an ablation, not opinion. Run the golden set through each configuration and compare per language.

Config	What it tests
A. Dense only	Baseline you proposed
B. Dense + <code>tsv_simple</code>	Value of exact-token match
C. B + <code>tsv_lang</code>	Value of stemmed lexical match
D. Best of A-C + reranker	Value of reranking

Pick the simplest config within 2 points of the best on hit rate@8 and faithfulness. If you ever add translation, keep the original query as a retrieval input too and only adopt it if the ablation shows a gain.

Ingestion and document normalization

Inconsistent documents are fixed once at ingestion, so retrieval sees one uniform shape. Re-ingest the full corpus into a new `index_version`.

1. **Parse** each doc (Apache Tika or LangChain4j document parsers) and detect its language (Lingua).

2. **Classify** `doc_type` : standalone, or part of a set (series, annex, FAQ split across files). Use metadata and file naming first; Gemma only for leftovers, with SME spot-checks.
3. **Structure-aware split**: by headings into sections (parents), then sections into chunks of roughly 300–500 tokens (children).
4. **Contextual header**: prepend `title > section path` (and the set name for part-of-set docs) to every chunk before embedding and FTS indexing, so a line like "the fee is 0.5%" still carries what it belongs to.
5. **Embed** with bge-m3 and build `tsv_simple` and `tsv_lang`.
6. **Exclude eval data**: historical Q&A pairs used as the golden set are never indexed, or are filtered out during eval runs.
7. **Validate**: count chunks per doc, flag empty or oversized chunks, and record the `index_version`.

At query time, retrieval matches on children and passes the parent section to the LLM, capped by a token budget.

Routing and tool layer

Routing is two-level and returns a typed decision. Level 1 picks the domain (RAG, RetroFees, future servers); level 2 picks the tool inside that domain via function calling. Adding a server adds one domain option, not ten new choices for the top router.

Decision contract (LangChain4j structured output into a sealed interface; enforce with JSON-schema constrained decoding in the Gemma serving layer)

```
sealed interface RouteDecision permits Route, MultiRoute, Clarify, OutOfScope {}
record Route(String domain, String tool, Map<String,Object> args, double confidence) implements RouteDecision {}
record MultiRoute(List<Route> routes) implements RouteDecision {}
record Clarify(String question, List<String> missingSlots, List<String> candidateTools) implements RouteDecision {}
record OutOfScope(String reason) implements RouteDecision {}
```

Tool registry entry (one per tool; loaded at startup, used to build router prompts and eval suites)

Field	Example
<code>name</code> , <code>domain</code>	<code>getRetroFeesByClient</code> , RetroFees
<code>description</code>	What it answers, in one paragraph

Field	Example
<code>requiredSlots</code>	client ID, date range
<code>positiveExamples</code>	5–10 questions it should handle
<code>negativeExamples</code>	Near-miss questions that belong to a neighbour tool
<code>entitlement</code>	Role or client scope required
<code>owner</code>	Team responsible
<code>evalSuite</code>	ID of its routing and answer test cases

Overlap handling, in order

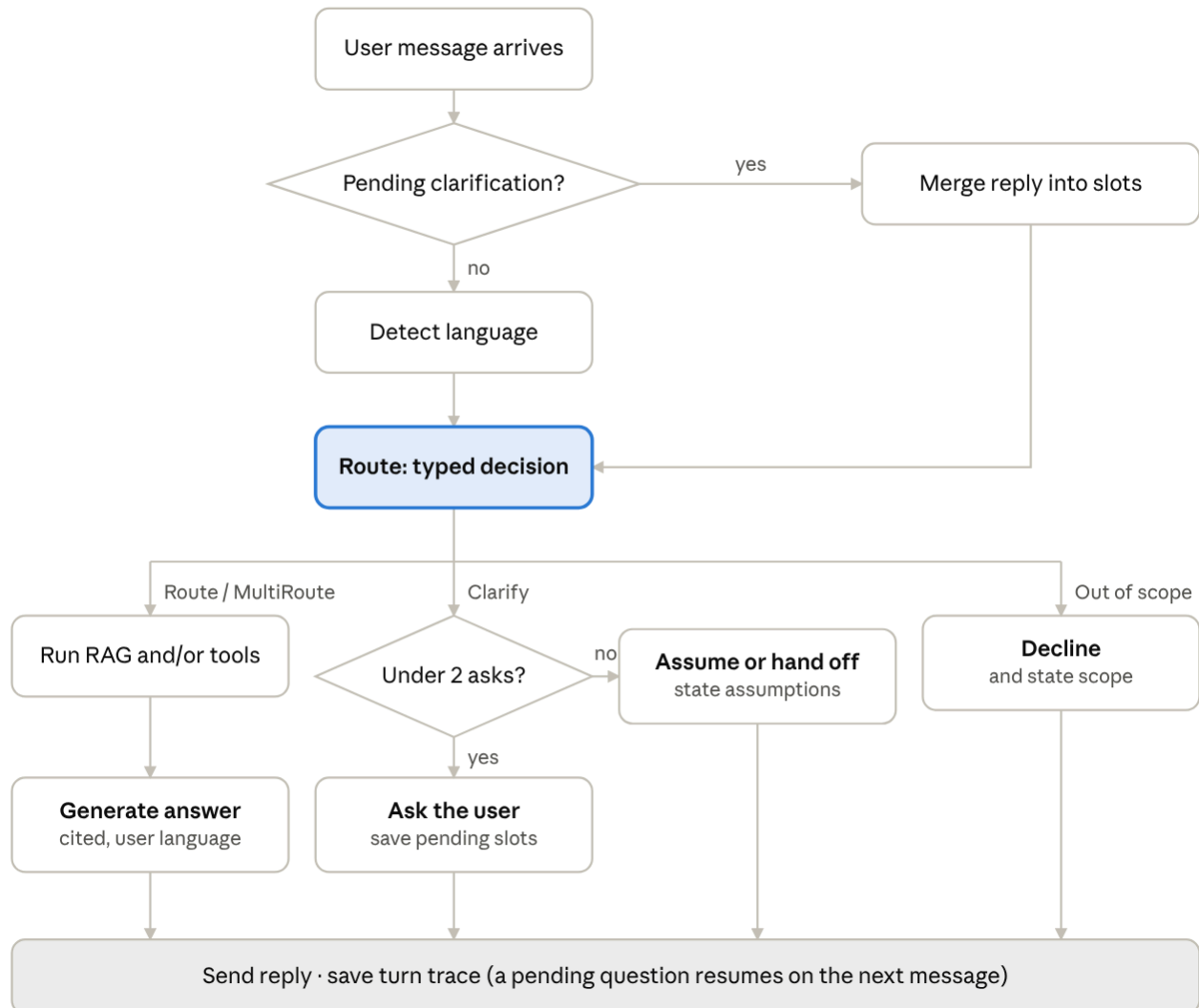
1. Sharpen contracts: negative examples that name the neighbour tool fix most overlap.
2. Merge tools that answer the same question shape into one parameterized tool.
3. Allow `MultiRoute` when a question genuinely spans tools; run in parallel, synthesize once.
4. Low margin between the top two candidates, or a missing required slot → `Clarify`.
5. Deterministic precedence rules as a last resort (for example, a question containing an account ID never routes to RAG alone).
6. Watch the routing confusion matrix in evals; any tool pair above an agreed confusion rate blocks release.

MCP servers connect through LangChain4j `McpToolProvider`; RetroFees tools use parameterized queries only, never LLM-generated SQL.

Functional workflow

Every turn checks for a pending clarification first, then routes once, and ends in exactly one of four outcomes.

Every turn ends in one of four outcomes: answer, clarify, assume, or decline



functional workflow · one turn, 4 outcomes

Step details

1. **Pending check.** Load conversation state by session (chat) or thread ID (email). If a clarification is pending, merge the user's reply into its slots and re-route the original question plus the new answer.
2. **Detect language.** Lingua sets the answer language and the FTS analyzer; nothing is translated.
3. **Route.** One Gemma call returns a `RouteDecision`. Clarify when a required slot is missing, when the top two tools are close, or when an entity is ambiguous.
4. **Execute.** RAG retrieval and tool calls run in parallel for a `MultiRoute`. Tool errors return a typed failure, never a guessed value.
5. **Generate.** Gemma answers in the user's language from retrieved chunks and tool results only, citing chunk IDs and tool names. If evidence is insufficient, it says so and offers a human handoff instead of guessing.

6. **Clarification cap.** At most two questions per user question; then answer with stated assumptions (for example, assuming the last 3 months) or hand off.
7. **Reply and trace.** Send the reply and store the `TurnTrace` for monitoring and evals.

Future: client servicing email channel

Email plugs in as a second channel adapter on the same orchestrator; the bot drafts and a servicing agent approves. Nothing in routing, retrieval or tools changes.

1. **Listen.** Microsoft Graph subscription (Exchange) or IMAP poller on the servicing mailbox; each email goes onto a queue, so processing is asynchronous and retryable.
2. **Adapt.** Parse body and attachments, attach to the existing thread ID, strip signatures and quoted history, mask PII, run prompt-injection checks.
3. **Identify.** Map the sender to a client and entitlement scope; unknown senders go straight to a human.
4. **Split.** Gemma breaks the email into individual questions; each runs through the normal turn workflow.
5. **Compose.** Combine answers into one draft reply in the client's language, with sources listed for the reviewer.
6. **Clarify.** A clarification outcome becomes a question inside the draft, or a flag for the agent to resolve.
7. **Approve.** The servicing agent approves, edits or rejects; nothing is auto-sent in the first release.
8. **Learn.** Log the agent's edits; edited and rejected drafts become new golden-set cases after review.

Past servicing emails, if indexed later, go into a separate retrieval source with per-client access control, never into the policy document index.

Evaluation framework (Java only)

A JUnit 5 harness scores every layer separately, deterministic metrics first and Gemma-as-judge only where meaning must be judged. Results are compared against the previous baseline run, not fixed thresholds.

Harness components

Component	Built with
Test runner	JUnit 5 <code>@ParameterizedTest</code> , cases streamed from the eval tables

Component	Built with
Pipeline under test	Real orchestrator returning a <code>TurnTrace</code> (language, decision, tool calls + args, chunk IDs, answer)
Deterministic scorers	Plain Java
LLM-judge scorers	LangChain4j <code>AiServices</code> per metric returning <code>record JudgeResult(boolean pass, String reason) ; Rage4j</code> as an optional head start
Results store	<code>eval_run</code> and <code>eval_result</code> tables (run ID, git SHA, model, prompt version, <code>index_version</code>)
Report	HTML or Allure report: per-metric deltas vs baseline, per language, per tool

Metrics by layer

Layer	Metric	Scorer
Routing	Domain/tool accuracy, confusion matrix, arg exact match	Deterministic
Clarification	Asked when expected, right slot, no over-asking	Deterministic on decision type + slots
Retrieval	Hit rate@k, MRR, context recall vs expected chunk IDs	Deterministic
Tool answers	Numbers and dates match DB ground truth	Deterministic
Generation	Faithfulness (claim-level), answer relevancy, correctness vs reference	Gemma judge
Language	Answer language = question language	Lingua, deterministic
Consistency	Same facts across translated variants of a question	Gemma judge on claim overlap

Living with one local model as both answerer and judge

- Prefer binary, claim-level checks: extract claims from the answer, then judge each claim "supported by context: yes/no". Far more stable than 1-10 scores.
- Use reference-based judging (against SME-validated answers) wherever a reference exists.

- Judge at temperature 0 with a separate prompt; run each judge case twice and flag disagreements.
- Calibrate: SMEs label about 100 cases; report judge-human agreement and re-check it after any model or prompt change.

Turning stored Q&A into the golden set

1. Export, de-duplicate, and drop pairs older than the current policy versions.
2. SME review of a sample to confirm the answers are still correct.
3. Tag each case: expected route, language, expected source doc/chunk IDs (retrieve candidates once, SME confirms).
4. Add cases history lacks: tool-overlap, clarification needed, out-of-scope, cross-lingual, and translated variants into 3-4 extra languages.
5. Version the dataset; split into a 50-case smoke set (every PR) and the full set (nightly and before release).

Non-functional requirements

Payment data and client emails make entitlements and audit non-negotiable from day one.

- **Entitlements:** the caller's identity (chat user or email sender) is passed to every tool; RetroFees enforces client scope inside the tool, not in the prompt. RAG chunks carry entitlement tags filtered in SQL.
- **Data safety:** parameterized queries only; PII masked in logs, traces and eval exports; prompt-injection checks on retrieved text and inbound email.
- **Observability:** OpenTelemetry spans per step (detect, route, retrieve, tool, generate), with the `TurnTrace` stored per turn; dashboards on latency, route mix, clarification rate, fallback rate.
- **Performance:** embedding and reranking run in-process (ONNX); the two retrieval legs run in one SQL round trip; Gemma calls are the latency budget, so cap the number per turn (router + optional tool-arg fill + generation).
- **Versioning:** model, prompt, tool registry and `index_version` recorded on every trace and eval run, so any answer can be reproduced.
- **Serving check:** confirm that the Gemma 4 serving stack supports function calling and JSON-schema constrained output before building on them; otherwise fall back to prompt-based JSON with schema validation and one retry.

Delivery plan and open decisions

Build the eval harness first, so every later change is measured against a baseline.

1. **Phase 1: Measure.** Golden set from stored Q&A, Java eval harness, `TurnTrace`, baseline on the current system. Exit: baseline report per language and route.
2. **Phase 2: Retrieval.** Re-ingest with normalization and context headers; run the A-D ablation; remove per-language branches. Exit: chosen config beats baseline on hit rate@8 and faithfulness in EN and FR.
3. **Phase 3: Routing.** Tool registry, two-level router, `RouteDecision`, clarification loop, RetroFees tools 2-10 each with eval cases. Exit: routing accuracy and confusion targets met.
4. **Phase 4: Email.** Channel adapter, queue, question splitting, draft-and-approve. Exit: SME acceptance rate of drafts on a pilot mailbox.

Open decisions

- ☐ Can a local reranker (bge-reranker-v2-m3) be hosted, or does retrieval stop at RRF?
- ☐ Are Postgres extensions for true BM25 (for example ParadeDB `pg_search`) allowed, or is built-in FTS final?
- ☐ Does the Gemma serving stack support function calling and constrained JSON output?
- ☐ Which email platform (Exchange via Microsoft Graph, or IMAP) and which pilot mailbox?
- ☐ Who are the SMEs for golden-set validation and judge calibration?
- ☐ Release targets for routing accuracy, confusion rate and faithfulness.